

LegalEase: AI-Powered Legal Document Generator

Project Design Phase

1. Team Details

Team Name: LegalEase: AI-Powered Legal Document generator

Team Members:

- 1. Bharathkumar A
- 2. Gurusankar S
- 3. Bikash Thaabaa R
- 4. Sabarivasan S

2. Introduction

The Project Design phase defines the overall structure, modules, data flow, user interface, and technical architecture of LegalEase. The design is prepared based on the requirements identified during the Requirement Analysis phase.

3. System Architecture

LegalEase follows a three-layer architecture consisting of the Streamlit frontend, FastAPI backend, and Gemini AI core. The frontend collects user information, the backend processes API requests, and the AI core generates the legal document draft.

4. Frontend Design

- Streamlit is used to create the user interface.
- The user can select the required document type.
- Input fields are provided for parties, terms and conditions, and effective date.
- A Generate button sends the information to the backend.
- The generated document is displayed for preview and editing.
- Download options are provided for PDF and Word documents.

5. Backend Design

- FastAPI is used to implement the backend service.
- main.py initializes the FastAPI application.
- routes.py defines the /generate API endpoint.
- Pydantic is used for request data validation.
- The backend sends the validated information to the Gemini document generator.

6. AI Module Design

The AI module is implemented in `ai_core/gemini_generator.py`. It receives the document type, parties, terms, and effective date, creates a structured prompt, and sends it to the Google Gemini model. The generated legal document draft is returned to the backend.

7. Document Processing Design

- Generated text is displayed in an editable preview.
- The edited content can be converted into a Word document.
- The content can be converted into a PDF document.
- Generated files are made available through download buttons.

8. Data Flow Design

- User enters document information in the Streamlit interface.
- Streamlit sends the information to the FastAPI `/generate` endpoint.
- FastAPI validates the request.
- The Gemini AI module generates the document.
- The generated document is returned to Streamlit.
- The user previews and edits the document.
- The final document can be downloaded.

9. Project Module Structure

- `app.py` – Streamlit frontend
- `main.py` – FastAPI application
- `routes.py` – API routes and request model
- `ai_core/gemini_generator.py` – Gemini AI document generation
- `output/` – generated documents
- `assets/` – project assets
- `.env` – API configuration

10. User Interface Design

The interface is designed to be simple and easy to understand. It contains document type selection, input fields, document generation controls, a generated-document preview, an editing area, and download options.

11. Security Design

The Gemini API key is stored in an environment variable instead of being written directly into source code. The `.env` file should not be uploaded to a public GitHub repository. User-provided information should also be handled carefully.

12. Design Outcome

The Project Design phase establishes a clear architecture and module structure for LegalEase. The design connects the Streamlit frontend, FastAPI backend, Gemini AI module, and document export functions into a single workflow for generating editable legal document drafts.